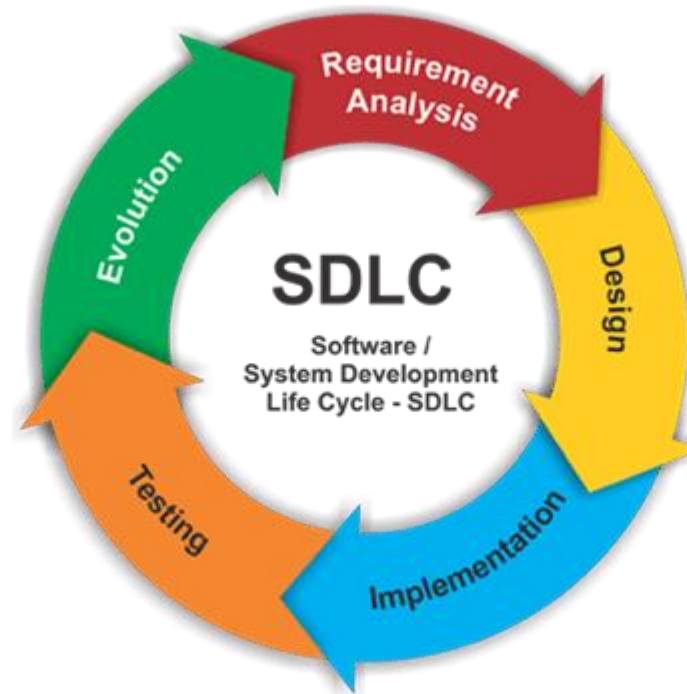


SDLC MODELS



Iterative Waterfall Model

- ▶ **Classical waterfall model is idealistic:**
 - ▶ assumes that no defect is introduced during any development activity.
 - ▶ in practice:
 - ▶ defects do get introduced in almost every phase of the life cycle.

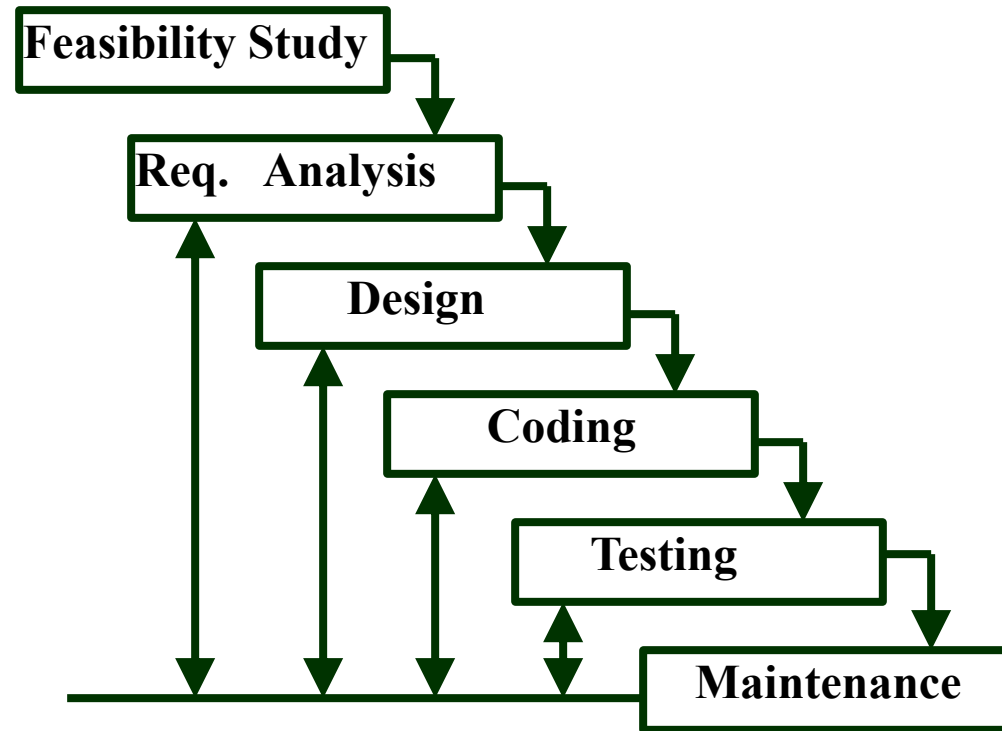
Iterative Waterfall Model (CONT.)

- ▶ **Defects usually get detected much later in the life cycle:**
 - ▶ For example, a design defect might go unnoticed till the coding or testing phase.

Iterative Waterfall Model (CONT.)

- ▶ Once a defect is detected:
 - ▶ we need to go back to the phase where it was introduced
 - ▶ redo some of the work done during that and all subsequent phases.
- ▶ Therefore we need feedback paths in the classical waterfall model.

Iterative Waterfall Model (CONT.)



Iterative Waterfall Model (CONT.)

- ▶ **Errors should be detected**
 - in the same phase in which they are introduced.
- ▶ **For example:**
 - if a design problem is detected in the design phase itself,
 - the problem can be taken care of much more easily than, if it is identified at the end of the integration and system testing phase.

Phase containment of errors

- ▶ The principle of detecting errors as close to its point of introduction as possible:
 - ▶ is known as **phase containment of errors**.
- ▶ Iterative waterfall model is most widely used model.
 - ▶ Almost every other model is derived from the waterfall model.

Prototyping model

- ▶ The basic idea in **Prototype model** is that instead of freezing the requirements before a design or coding can proceed, a throwaway **prototype** is built to understand the requirements.
- ▶ This **prototype** is developed based on the currently known requirements.

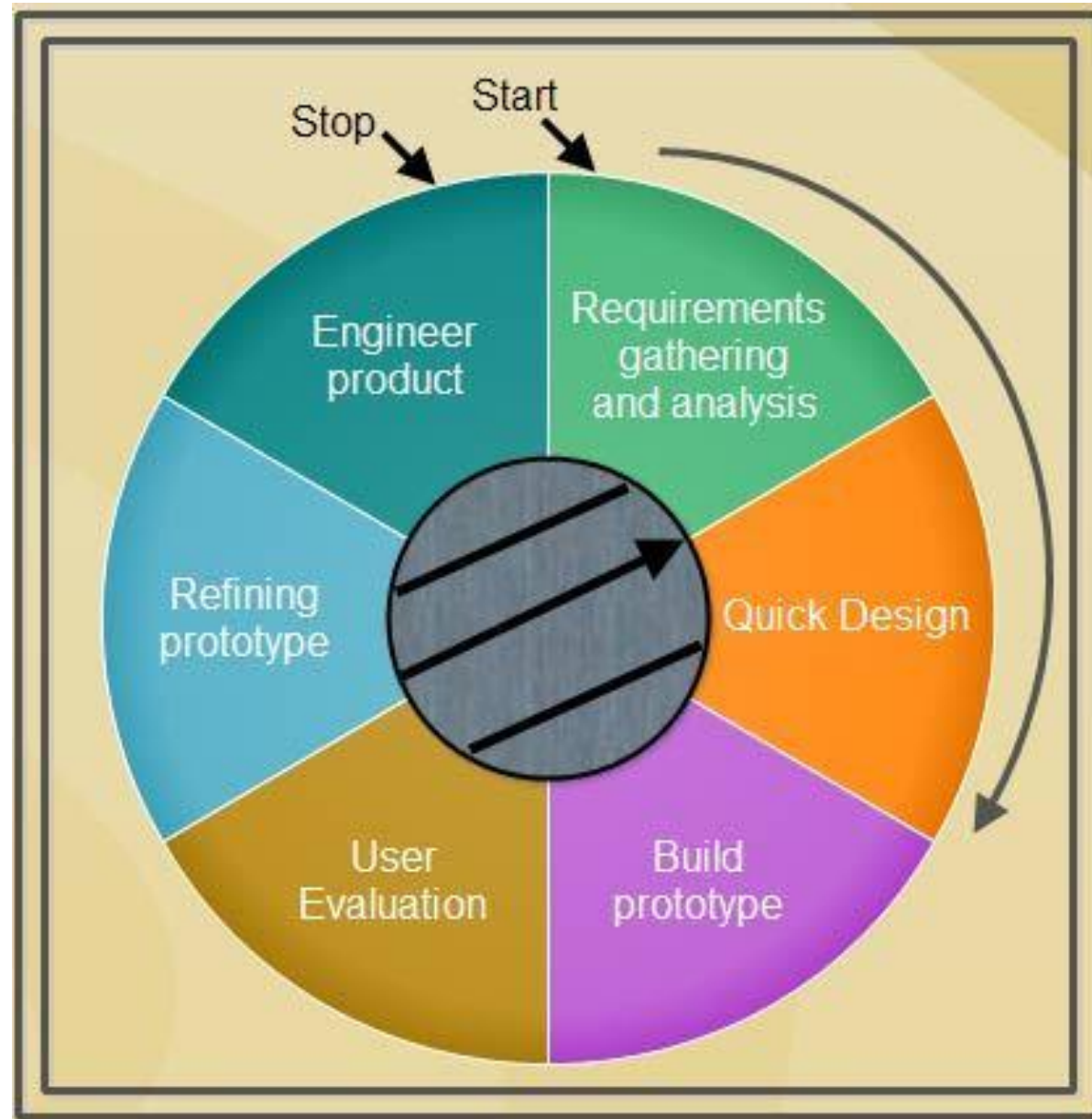
Prototyping Model

- ▶ Before starting actual development,
 - ▶ a working prototype of the system should first be built.
- ▶ A prototype is a toy implementation of a system:
 - ▶ limited functional capabilities,
 - ▶ low reliability,
 - ▶ inefficient performance.

- ▶ **Prototype model**
- ▶ An *early approximation of a final system or product*) is built, tested, and then reworked as necessary until an acceptable prototype is finally achieved.
- ▶ The **prototyping model** is applied when detailed information related to input and output requirements of the system is not available. In this model, it is assumed that all the requirements may not be known at the start of the development of the system.

Prototyping Model (CONT.)

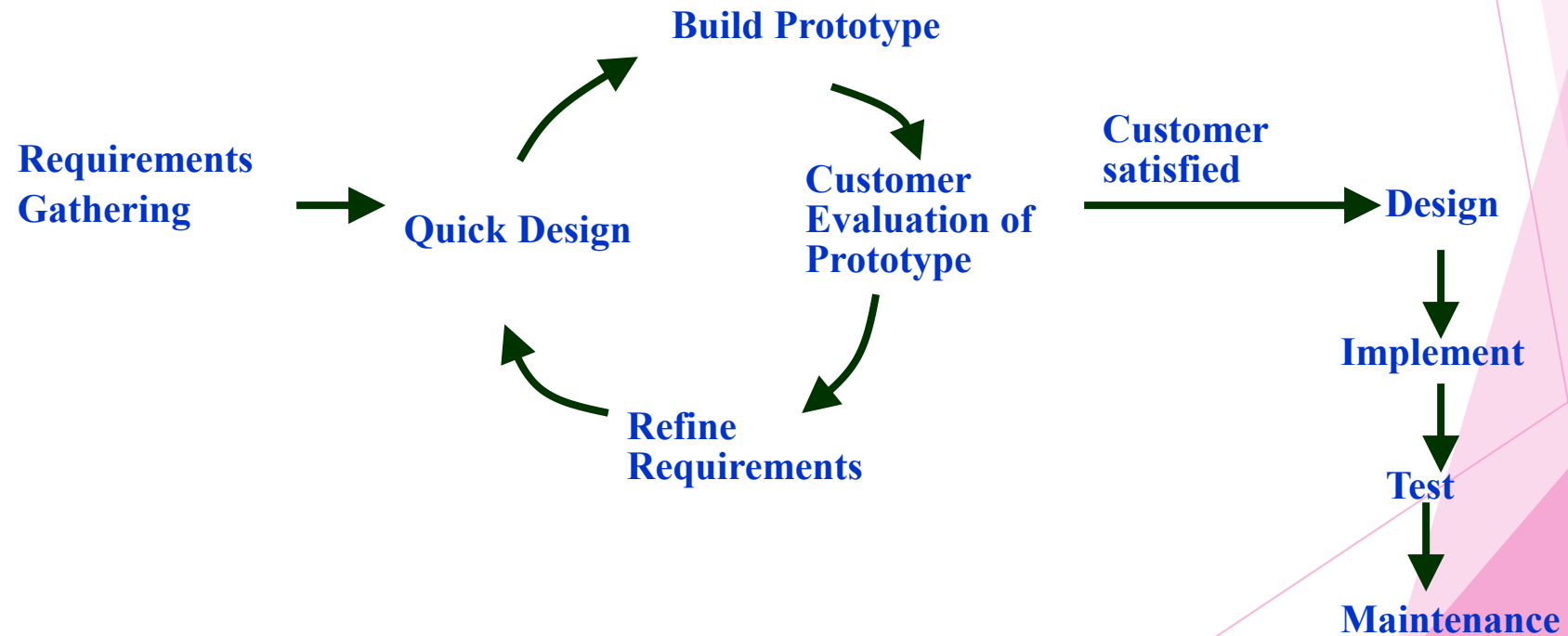
- ▶ The reason for developing a prototype is:
 - ▶ it is impossible to "get it right" the first time,
 - ▶ we must plan to throw away the first product
 - ▶ if we want to develop a good product.



Prototyping Model (CONT.)

- ▶ **The developed prototype is submitted to the customer for his evaluation:**
 - ▶ Based on the user feedback, requirements are refined.
 - ▶ This cycle continues until the user approves the prototype.
- ▶ **The actual system is developed using the classical waterfall approach.**

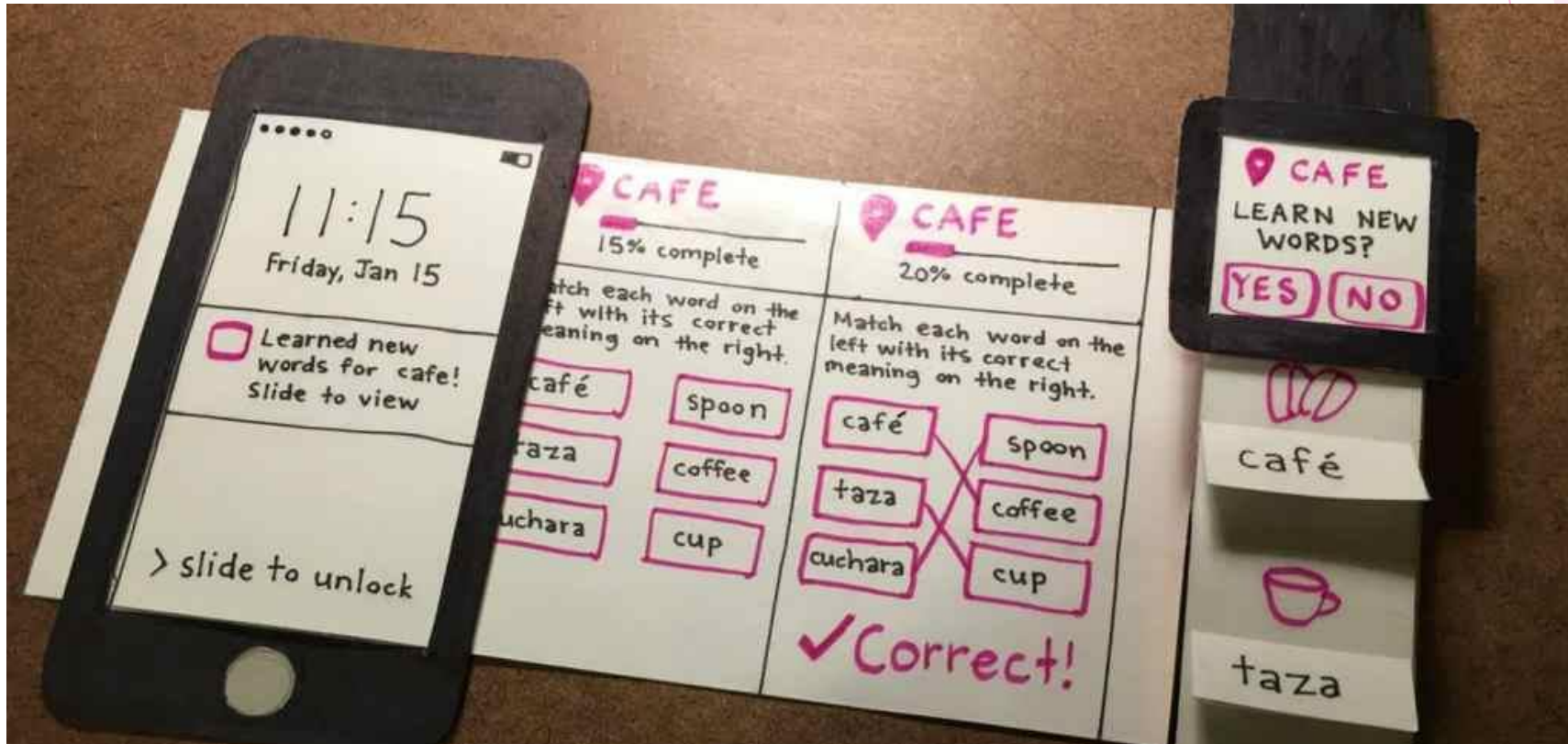
Prototyping Model (CONT.)



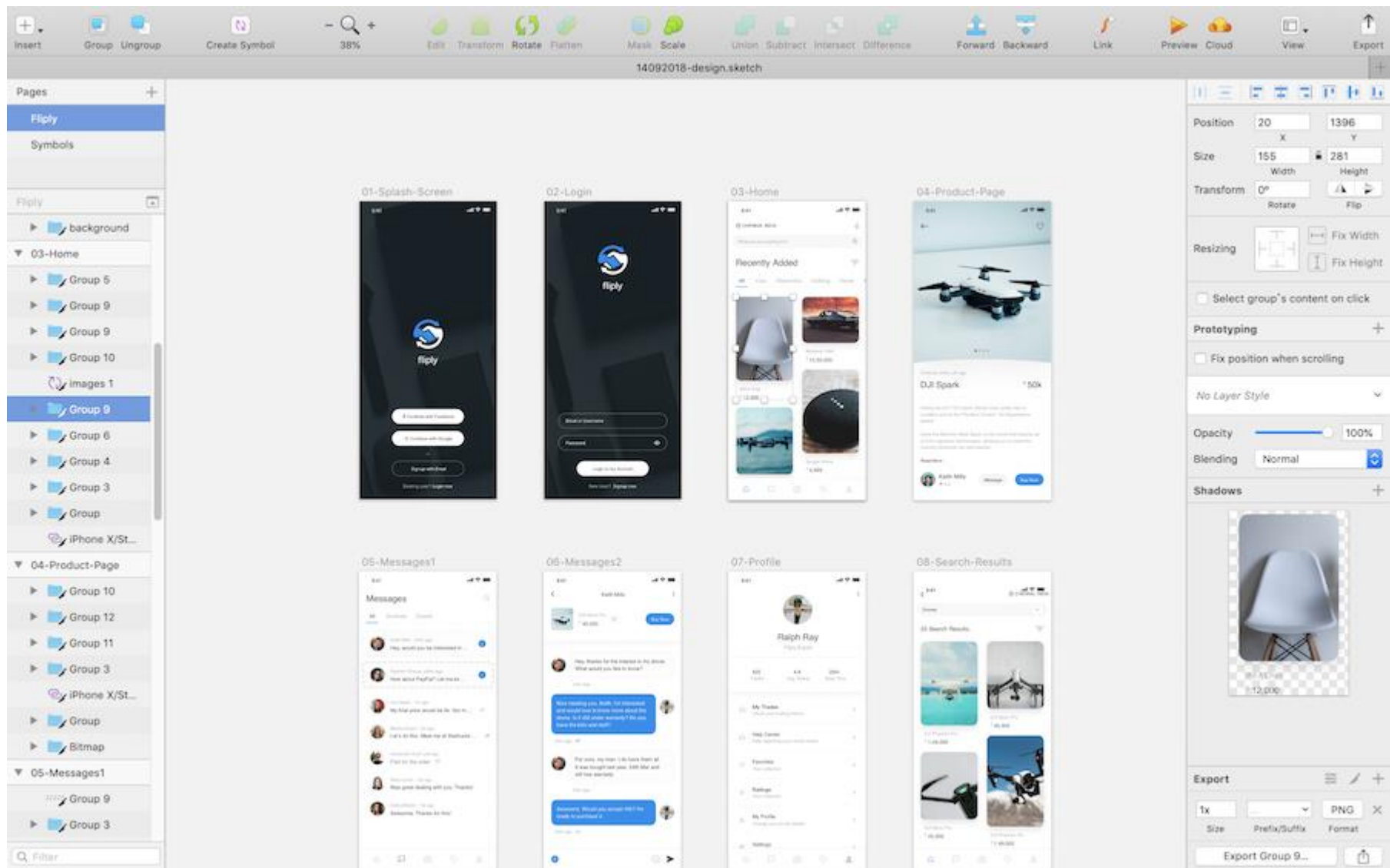
Prototyping Model (CONT.)

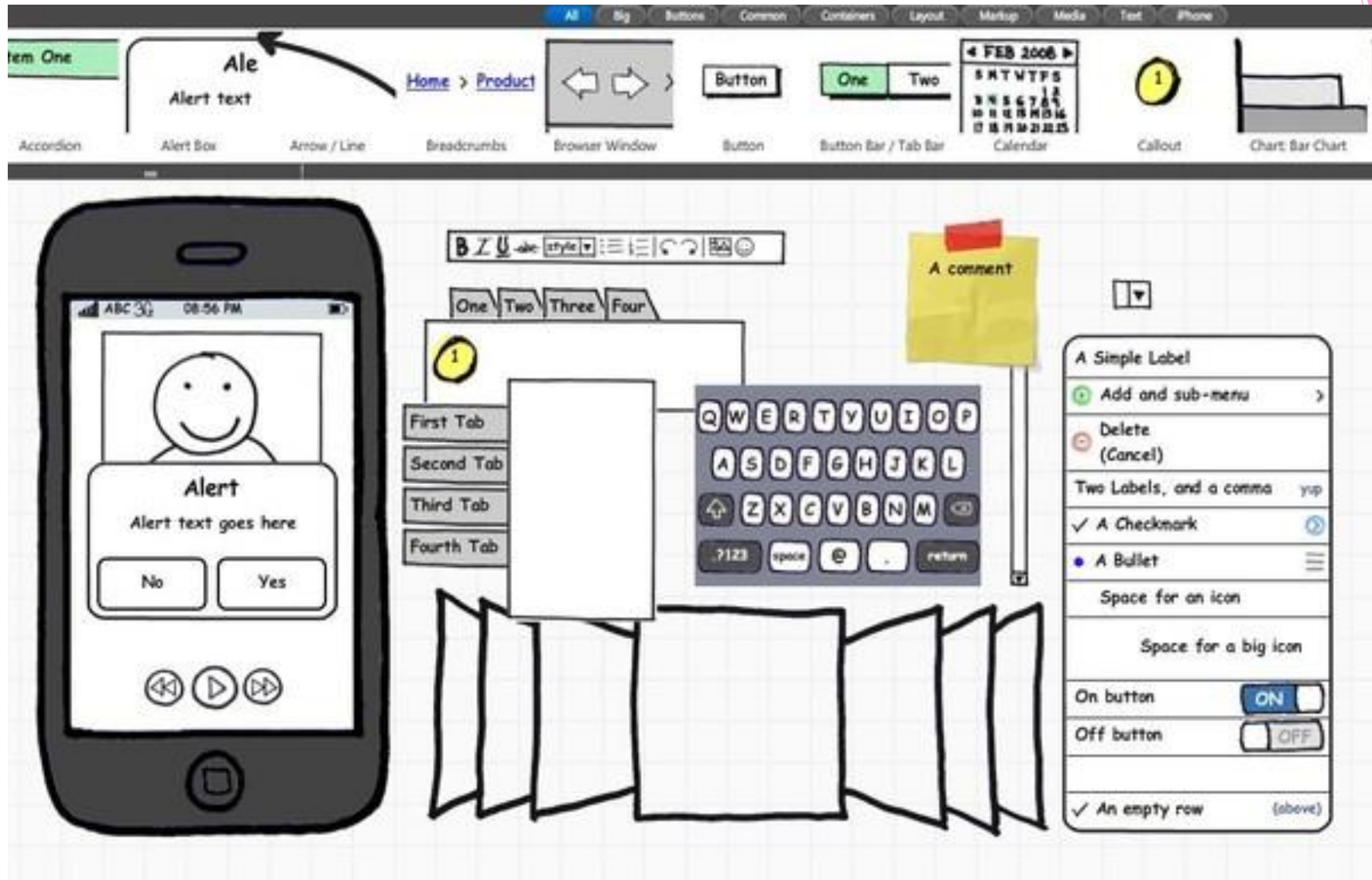
- ▶ Requirements analysis and specification phase becomes redundant:
 - ▶ final working prototype (with all user feedbacks incorporated) serves as an **animated requirements specification**.
- ▶ Design and code for the prototype is usually thrown away:
 - ▶ However, the experience gathered from developing the prototype helps a great deal while developing the actual product.

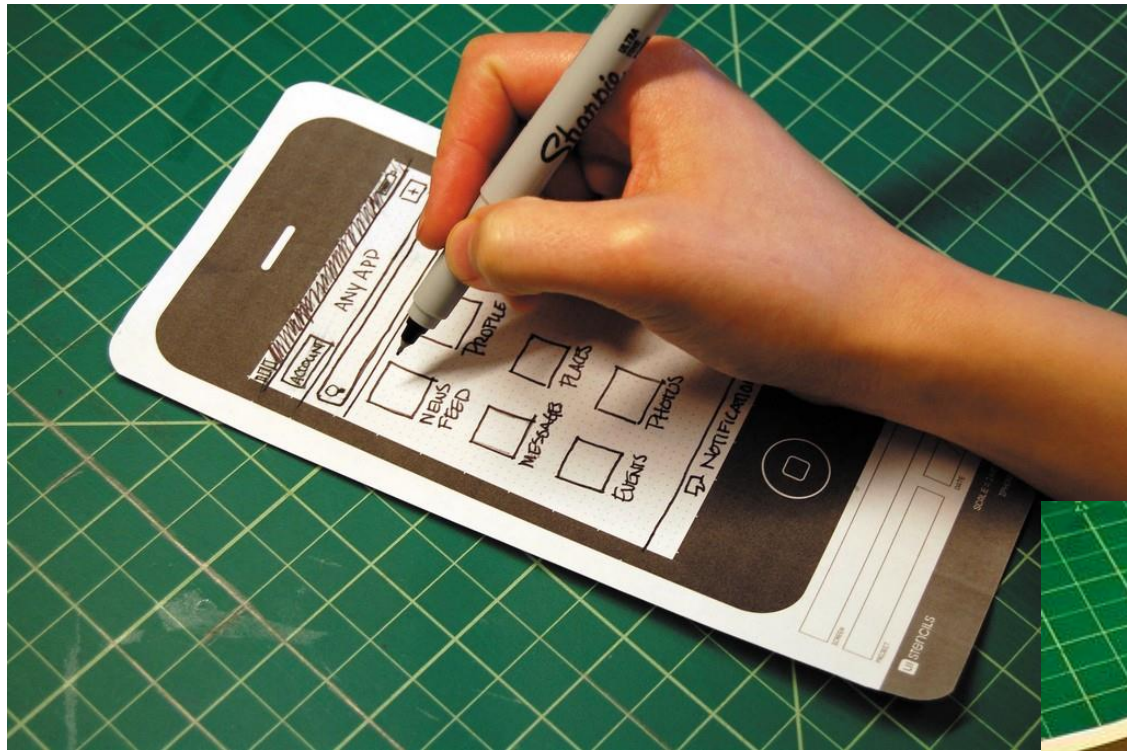
Paper prototype is an example of throwaway prototype created in the form of rough or hand-sketched drawings of the product's interface, front-end design, and sometimes the back end work.











Prototyping Model (CONT.)

- ▶ **Even though construction of a working prototype model involves additional cost --- overall development cost might be lower for:**
 - ▶ systems with unclear user requirements,
 - ▶ systems with unresolved technical issues.
- ▶ **Many user requirements get properly defined and technical issues get resolved:**
 - ▶ these would have appeared later as change requests and resulted in incurring massive redesign costs.

Advantages

- ▶ Users are actively involved in the development
 - ▶ □ Since in this methodology a working model of the system is provided, the users get a better understanding of the system being developed.
 - ▶ □ Errors can be detected much earlier.
 - ▶ □ Quicker user feedback is available leading to better solutions.
 - ▶ □ Missing functionality can be identified easily
 - ▶ □ Confusing or difficult functions can be identified
- Requirements validation, Quick implementation of incomplete, but functional application..

Disadvantages of Prototype model:

- ▶ 1. Leads to implementing and then repairing of building systems.
- ▶ 2. Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.
- ▶ 3. Incomplete application may cause application not to be used as the full system was designed.
- ▶ 4. Incomplete or inadequate problem analysis.

When to use Prototype model:

- ▶ Prototype model should be used when the desired system needs to have a lot of interaction with the end users.
- ▶ Typically, online systems, web interfaces have a very high amount of interaction with end users, are best suited for Prototype model.
- ▶ Prototyping ensures that the end users constantly work with the system and provide a feedback which is incorporated in the prototype to result in a useable system.
- ▶ They are excellent for designing good human computer interface systems

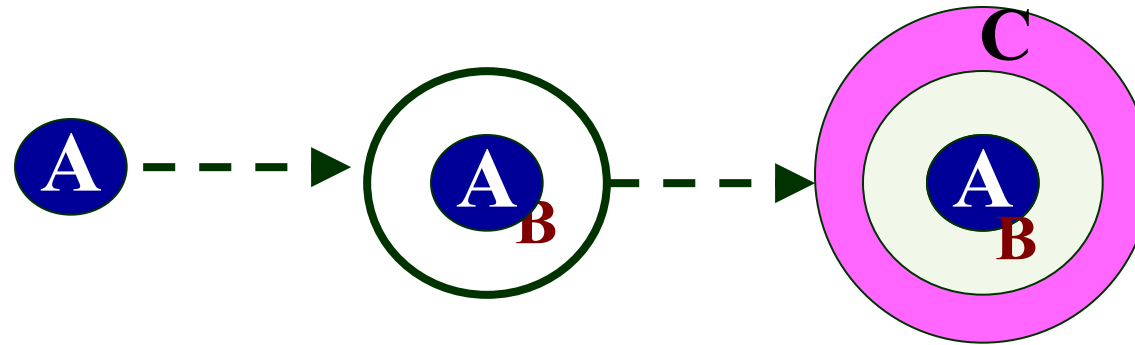
Evolutionary Model

- ▶ **Evolutionary model:**
 - ▶ The system is broken down into several modules which can be incrementally implemented and delivered.
- ▶ First develop the core modules of the system.
- ▶ The initial product skeleton is refined into increasing levels of capability:
 - ▶ by adding new functionalities in successive versions.

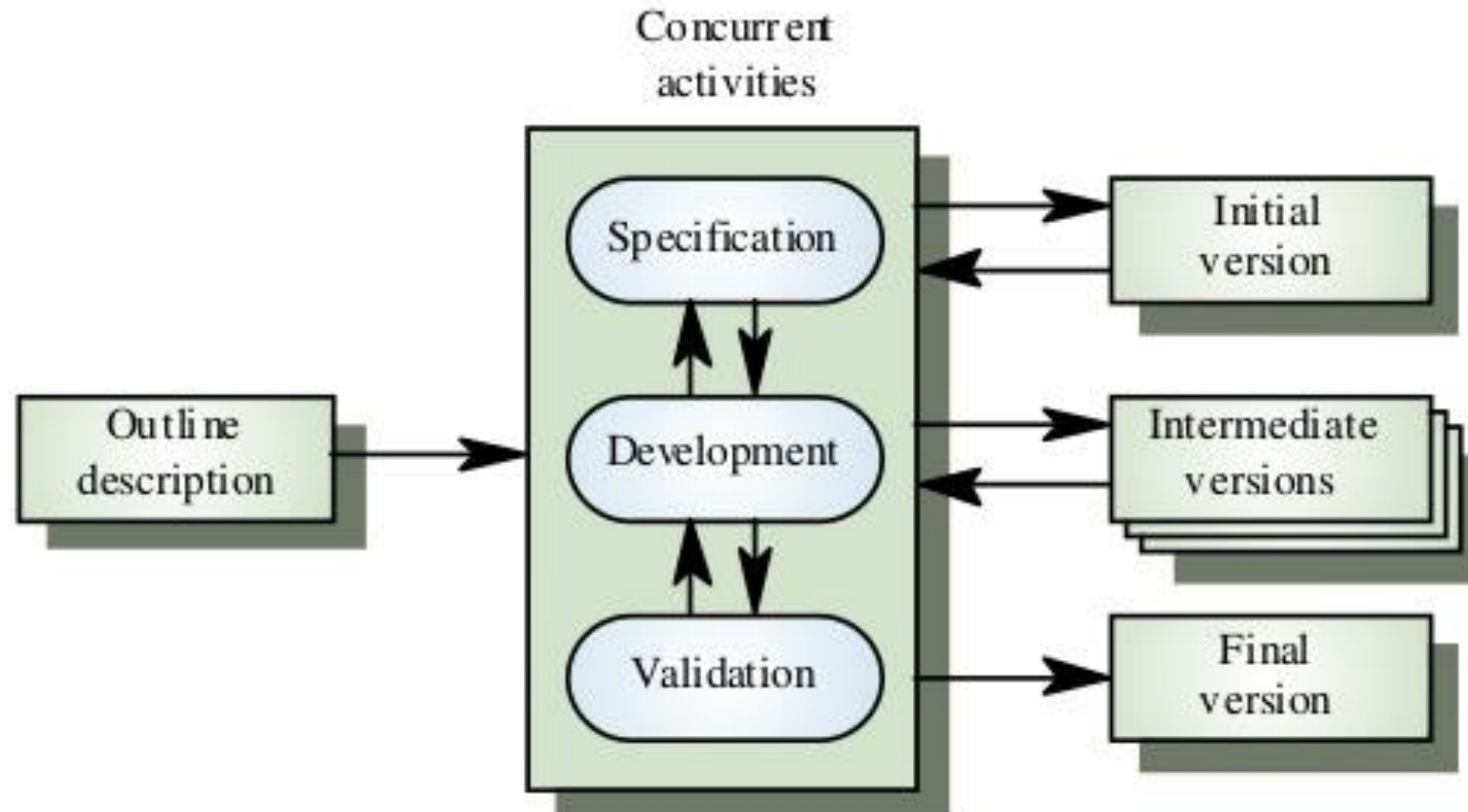
Evolutionary Model (CONT.)

- ▶ **Successive version of the product:**
 - ▶ **functioning systems capable of performing some useful work.**
 - ▶ **A new release may include new functionality:**
 - ▶ **also existing functionality in the current release might have been enhanced.**

Evolutionary Model (CONT.)



Evolutionary Process Model



Advantages of Evolutionary Model

- ▶ Users get a chance to experiment with a partially developed system:
 - ▶ much before the full working version is released,
- ▶ Helps finding exact user requirements:
 - ▶ much before fully working system is developed.
- ▶ Core modules get tested thoroughly:
 - ▶ reduces chances of errors in final product.

Disadvantages of Evolutionary Model

- ▶ Often, difficult to subdivide problems into functional units:
 - ▶ which can be incrementally implemented and delivered.
- ▶ evolutionary model is useful for very large problems,
 - ▶ where it is easier to find modules for incremental implementation.

Evolutionary Model with Iteration

- ▶ Many organizations use a combination of iterative and incremental development:
 - ▶ a new release may include new functionality
 - ▶ existing functionality from the current release may also have been modified.

Evolutionary Model with iteration

- ▶ **Several advantages:**
 - ▶ **Training can start on an earlier release**
 - ▶ customer feedback taken into account
 - ▶ **Markets can be created:**
 - ▶ for functionality that has never been offered.
 - ▶ **Frequent releases allow developers to fix unanticipated problems quickly.**

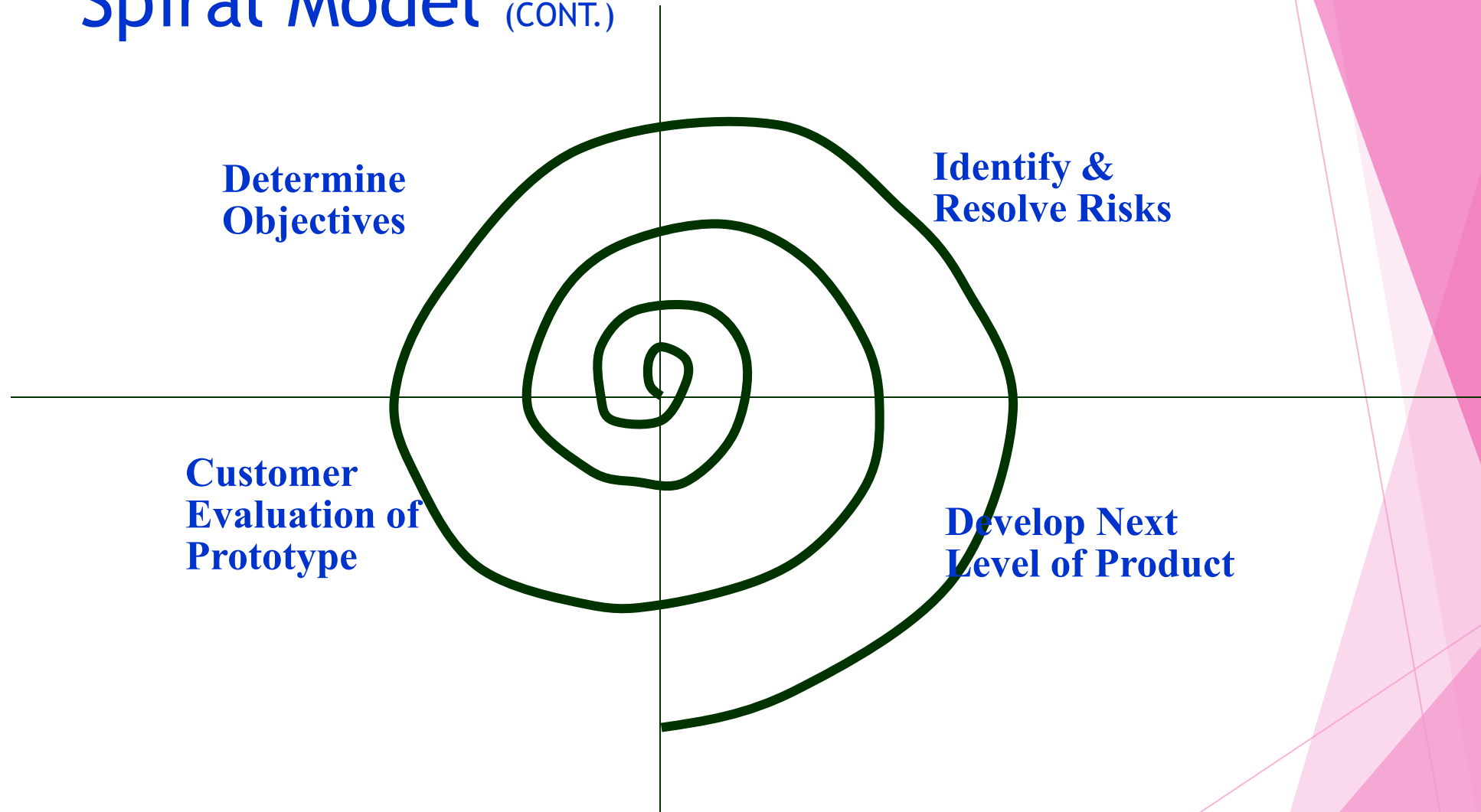
Spiral Model

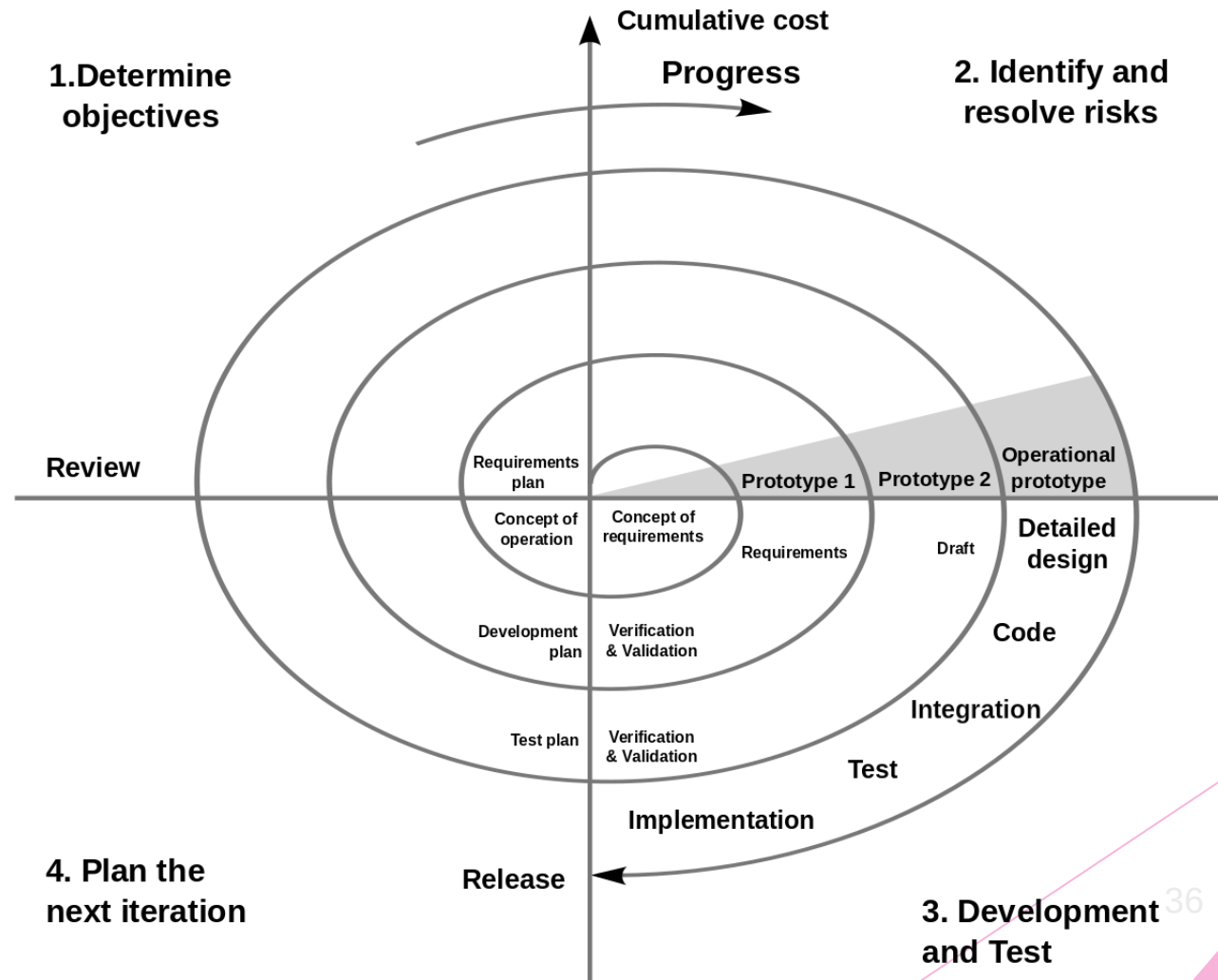
- ▶ Proposed by Boehm in 1988.
- ▶ Each loop of the spiral represents a phase of the software process:
 - ▶ the innermost loop might be concerned with system feasibility,
 - ▶ the next loop with system requirements definition,
 - ▶ the next one with system design, and so on.
- ▶ There are no fixed phases in this model, the phases shown in the figure are just examples.

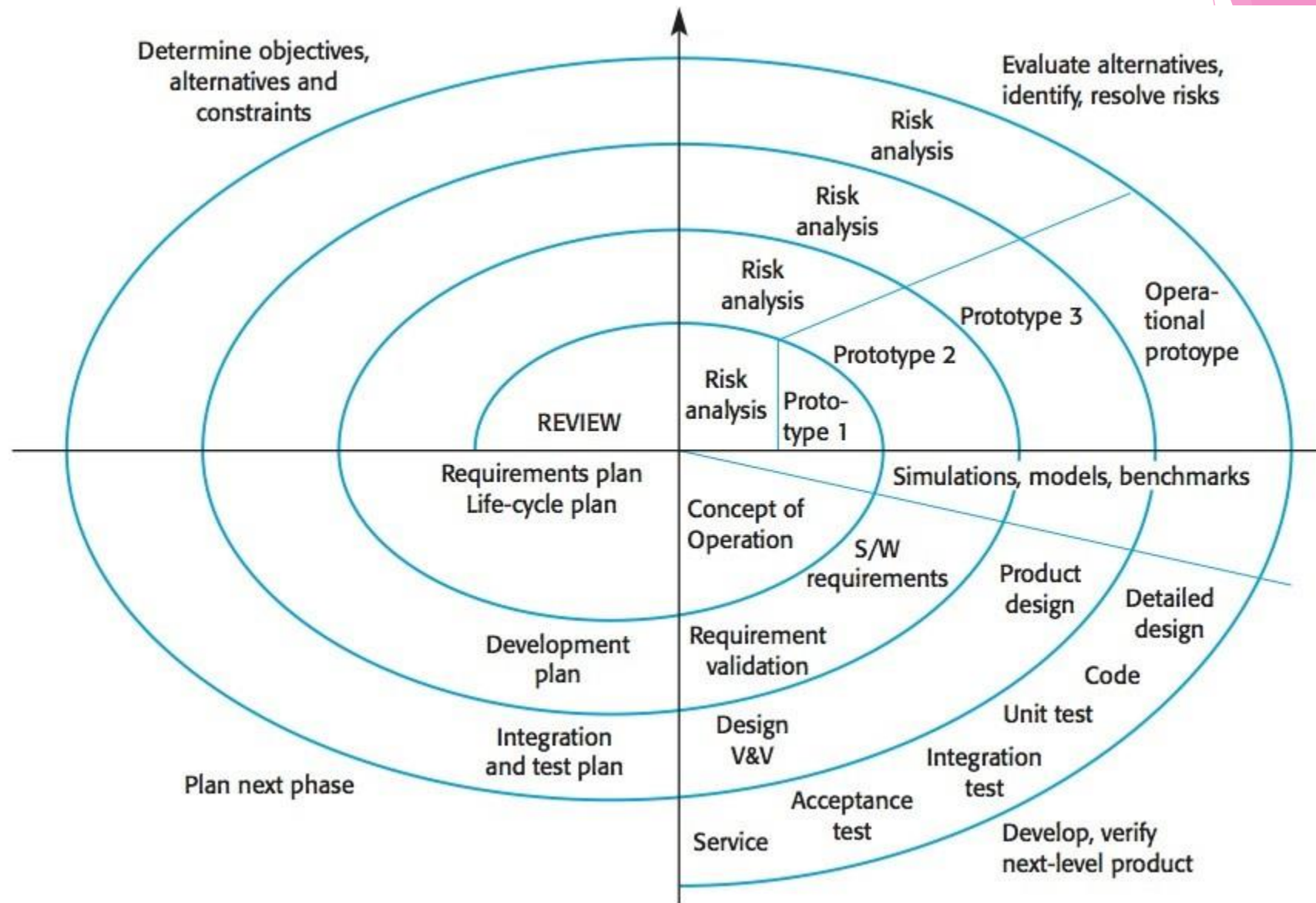
Spiral Model (CONT.)

- ▶ The team must decide:
 - ▶ how to structure the project into phases.
- ▶ Start work using some generic model:
 - ▶ add extra phases
 - ▶ for specific projects or when problems are identified during a project.
- ▶ Each loop in the spiral is split into four sectors (quadrants).

Spiral Model (CONT.)







Schedule Risk:

- ▶ Project schedule get slip when project tasks and schedule release risks are not addressed properly. Schedule risks mainly affect a project and finally on company economy and may lead to project failure. **Schedules often slip due to the following reasons:**

- ▶ Wrong time estimation
- ▶ Resources are not tracked properly. All resources like staff, systems, skills of individuals etc.
- ▶ Failure to identify complex functionalities and time required to develop those functionalities.
- ▶ Unexpected project scope expansions.

- ▶ **Budget Risk:**
- ▶ Wrong budget estimation.
- ▶ Cost overruns
- ▶ Project scope expansion
- ▶ **Operational Risks:**
Risks of loss due to improper process implementation failed system or some external events risks.
Causes of Operational risks:
 - ▶ Failure to address priority conflicts
 - ▶ Failure to resolve the responsibilities
 - ▶ Insufficient resources
 - ▶ No proper subject training
 - ▶ No resource planning
 - ▶ No communication in the team.

► **Technical risks:**

Technical risks generally lead to failure of functionality and performance.
Causes of technical risks are:

- Continuous changing requirements
- No advanced technology available or the existing technology is in initial stages.
- The product is complex to implement.
- Difficult project modules integration.

▶ **Programmatic Risks:**

These are the external risks beyond the operational limits. These are all uncertain risks are outside the control of the program.

These external events can be:

- ▶ Running out of the fund.
- ▶ Market development
- ▶ Changing customer product strategy and priority
- ▶ Government rule changes.

Objective Setting (First Quadrant)

- ▶ Identify objectives of the phase,
- ▶ Examine the **risks** associated with these objectives.
 - ▶ Risk:
 - ▶ any adverse circumstance that might hamper successful completion of a software project.
- ▶ Find alternate solutions possible.

Risk Assessment and Reduction (Second Quadrant)

- ▶ For each identified project risk,
 - ▶ a detailed analysis is carried out.
- ▶ Steps are taken to reduce the risk.
- ▶ For example, if there is a risk that the requirements are inappropriate:
 - ▶ a prototype system may be developed.

Spiral Model (CONT.)

- Development and Validation (**Third quadrant**):
 - develop and validate the next level of the product.
- Review and Planning (**Fourth quadrant**):
 - review the results achieved so far with the customer and plan the next iteration around the spiral.
- With each iteration around the spiral:
 - progressively more complete version of the software gets built.

Spiral Model as a Meta Model

- ▶ Subsumes all discussed models:
 - ▶ a single loop spiral represents waterfall model.
 - ▶ uses an evolutionary approach --
 - ▶ iterations over the spiral are evolutionary levels.
 - ▶ enables understanding and reacting to risks during each iteration along the spiral.
- ▶ Uses:
 - ▶ prototyping as a risk reduction mechanism
 - ▶ retains the step-wise approach of the waterfall model.

Comparison of Different Life Cycle Models

- ▶ **Iterative waterfall model**
 - ▶ most widely used model.
 - ▶ But, suitable only for well-understood problems.
- ▶ **Prototype model is suitable for projects not well understood:**
 - ▶ user requirements
 - ▶ technical aspects

Comparison of Different Life Cycle Models

(CONT.)

- ▶ **Evolutionary model is suitable for large problems:**
 - ▶ can be decomposed into a set of modules that can be incrementally implemented,
 - ▶ incremental delivery of the system is acceptable to the customer.
- ▶ **The spiral model:**
 - ▶ suitable for development of technically challenging software products that are subject to several kinds of risks.

Advantages of Spiral model:

- ▶ High amount of risk analysis hence, avoidance of Risk is enhanced.
- ▶ Good for large and mission-critical projects.
- ▶ Strong approval and documentation control.
- ▶ Additional Functionality can be added at a later date.
- ▶ Software is produced early in the software life cycle.
- ▶

Disadvantages of Spiral model:

- ▶ Can be a costly model to use.
- ▶ Risk analysis requires highly specific expertise.
- ▶ Project's success is highly dependent on the risk analysis phase.
- ▶ Doesn't work well for smaller projects.

When to use Spiral model:

- ▶ When costs and risk evaluation is important
- ▶ For medium to high-risk projects
- ▶ Long-term project commitment unwise because of potential changes to economic priorities
- ▶ Users are unsure of their needs
- ▶ Requirements are complex
- ▶ New product line
- ▶ Significant changes are expected (research and exploration)